Relatório Final — Transferência confiável sobre UDP: de Stop-and-Wait a RTT adaptativo

CIC0236 – Teleinformática e Redes 2 – Turma 01 – 2026/2 – Universidade de Brasília

Grupo 10 (`--group grupo10`)
Anna Luiza Novaes Jansen Silva – 232024572
Vinicius Henrique Barbosa Silverio de Castro – 232006832

1 Introdução e evolução do projeto

O UDP não garante entrega, ordem nem integridade. O servidor (`137.131.178.229:8080`/UDP) é *pull-based*: só responde a pedidos (`REQ`), então toda a confiabilidade (detecção de perda, retransmissão, ordenação e verificação por MD5) tem de existir no cliente. A sessão é sempre a mesma:

`HELLO` → `OK` (tamanho e MD5) → `REQ`, `DATA` e `ACK` por segmento → `FIN` (MD5 calculado) → `DONE|ok=1`

O projeto evoluiu em três etapas, cada uma motivada por uma limitação medida na anterior.

1.1 Entrega 1 — Stop-and-Wait (SAW)

O primeiro cliente fazia o ciclo mais simples possível: `HELLO`/`OK`, depois, para cada segmento, `REQ|seq=n`, espera do `DATA` com o mesmo `seq` até o timeout fixo de 500 ms, `ACK` e próximo segmento, terminando com `FIN`/`DONE` e conferência do MD5. Duas decisões de implementação evitaram erros que só apareceriam no MD5: o `DATA` é separado com `split(b"|", 3)`, porque o payload é binário e pode conter o byte `|`; e `DATA` com `seq` diferente do esperado (duplicata atrasada de uma retransmissão) é descartado sem reiniciar o prazo.

Os três arquivos foram recebidos íntegros (13/10/2026, sem perda simulada):

Tabela 1: Entrega 1: SAW com timeout fixo de 500 ms.

Arquivo	Tempo total	RTT médio	Throughput medido	Teórico ($512 \cdot 8/RTT$)	Retransmissões
small (1 MB)	131,9 s	38,49 ms	63,62 kbps	106,41 kbps	105
medium (5 MB)	588,7 s	39,17 ms	71,25 kbps	104,56 kbps	370
large (10 MB)	965,9 s	35,58 ms	86,84 kbps	115,13 kbps	465

O resultado mais útil da Entrega 1 foi a decomposição do tempo: o tempo total é explicado quase exatamente por $N \times RTT_{\text{médio}}$ + retransmissões × 0,5 s (a sobra foi de 0,6 a 4,7 s, menos de 0,5 %). Descontando os timeouts, o SAW chega a 104–114 kbps, o teto de um segmento por RTT. O que afastava o medido do teórico (60 % a 75 %) eram as perdas naturais da rede real (2,2 % a 4,9 % dos envios): cada perda parava o cliente por 500 ms, cerca de 13 RTTs; no small, 40 % do tempo foi espera de timeout. Dois fatos já apontavam para o que viria depois: a mediana do RTT era de ∼26–30 ms, de modo que o timeout fixo era 17 a 20 vezes maior que o necessário; e o limite de um segmento por RTT pedia mais de um segmento em trânsito.

Resposta ao feedback da Entrega 1. O professor apontou que a perda de 2,2–4,9 % era alta para um RTT mediano normal (∼26–30 ms) e perguntou quantos timeouts podem ter sido prematuros, já que o RTT máximo (473,9 ms) chegou perto do timeout de 500 ms.

- **Ambiente:** os três testes rodaram em sequência em 13/10/2026, das 21:15 às 21:44 (horário de pico do uso doméstico), a partir de Wi-Fi residencial, com o cliente atrás de NAT (IP `192.168.100.9`, visto na captura da Seção 4).
- **A perda veio do ambiente, não do código:** com o mesmo código e o mesmo timeout de 500 ms, o SAW com o arquivo large teve 772 retransmissões (3,6 %) às 18:32 de 10/11 e apenas 3 (0,015 %) às 19:23 do mesmo dia, com o RTT médio caindo de 35,3 para 26,9 ms. A perda natural varia muito com o horário; 13/10 foi um dia ruim.
- **Timeouts prematuros: praticamente nenhum.** Um timeout é prematuro quando o DATA original chega depois de 500 ms. Entre os 31 873 segmentos entregues sem retransmissão nos três arquivos, o RTT passou de 250 ms em só 12 e de 400 ms em só 2 (máximo de 473,9 ms, no medium); o percentil 99 ficou em 134–147 ms. Pela cauda da distribuição, atrasos acima de 500 ms seriam no máximo alguns poucos casos entre as 940 retransmissões. Além disso, um DATA atrasado que chegasse logo depois do reenvio apareceria no CSV com `rtt_ms` (medido a partir do reenvio) abaixo do RTT mínimo do caminho (∼19 ms); nos 895 segmentos retransmitidos o menor valor foi 19,9 ms. Os dados indicam que as retransmissões foram, quase todas, perdas reais. Na Entrega 3 a captura do Wireshark permitiu medir isso diretamente (Seção 4).

1.2 Entrega 2 — Janela deslizante: Go-Back-N (GBN) e Selective Repeat (SR)

Antes de tudo, o código foi refatorado: o que não depende do protocolo (pacotes, handshake, MD5, CSV, simulação de perda) foi para `comum.py`, e `saw.py`, `gbn.py` e `sr.py` passaram a implementar a mesma interface, `run(sess) -> (payloads, rows)`, com o estado em um dicionário `sess`. Cada protocolo só decide quando enviar `REQ` e o que fazer com cada `DATA`.

As dificuldades principais e como foram resolvidas:

- **Adaptar a janela a um servidor *pull-based*.** No livro, a janela anda com ACKs recebidos pelo remetente; aqui o cliente controla tudo e recebe `DATA`. Passamos a tratar "chegou o DATA da base" como o equivalente do "chegou o ACK da base".
- **Simular perda de modo que ela realmente cause retransmissão.** Descartar o ACK (abordagem b) não provoca timeout algum neste protocolo, pois o servidor só reage a `REQ`. Adotamos a abordagem (a): o `DATA` é descartado com probabilidade P dentro de `recv_data`, antes de qualquer processamento, com `random.Random(seed)` para repetibilidade (`--seed 42`) e o mesmo ponto de aplicação nos três protocolos.
- **Separar perda simulada de perda natural.** O CSV ganhou as colunas `simulated_loss`, `sim_drops` e `recv_time_ms`. Uma retransmissão sem perda simulada correspondente indica perda natural.
- **Um temporizador por segmento no SR com um único socket.** `recv_data` espera até o menor prazo entre os pendentes (`last_send[seq] + timeout`), e só os segmentos vencidos são reenviados. No GBN há um só temporizador, o da base, e o timeout reenvia a base e todos os seguintes.
- **Contar timeouts no GBN.** `timeout_occurred` marca só o segmento que estava na base; o número de eventos precisou ser inferido das retransmissões.

Todos os 21 cenários (SAW, GBN e SR × janelas 4/8/16 × 0 %, 5 % e 10 %, small, seed 42) terminaram com o MD5 correto. O resumo dos principais resultados:

Tabela 2: Entrega 2, arquivo small, timeout fixo: throughput (kbps) / retransmissões / tempo (s).

Protocolo	Sem perda	5 % de perda	10 % de perda
SAW	167,2 / 0 / 50,2	88,3 / 91 / 95,0	52,0 / 220 / 161,3
GBN-16	2 025,5 / 0 / 4,1	159,5 / 1 512 / 52,6	70,3 / 3 562 / 119,3
SR-16	1 900,7 / 0 / 4,4	260,7 / 91 / 32,2	153,7 / 220 / 54,6

Sem perda, a janela multiplica a vazão (∼12× com $N = 16$) e GBN e SR são equivalentes. Com perda, o quadro muda: o GBN reenvia a janela inteira a cada timeout (3 562 retransmissões contra 220 do

SR com 10 %, cerca de 16×) e seus timeouts de 500 ms se somam, enquanto no SR eles se sobrepõem (220 timeouts, mas só 98 paradas de mais de 300 ms). Uma varredura extra do tamanho da janela, sem perda, mostrou que a vazão estabiliza em ∼2 700–2 900 kbps a partir de $N \approx 32$–48, e que em $N = 128$ surgem perdas naturais que derrubam o GBN para ∼1 700 kbps. Uma lição de método: a rede varia de um dia para o outro (o SAW sem perda deu 167 kbps na Entrega 2 contra 64 kbps na Entrega 1, com RTT médio de ∼24 ms contra ∼38 ms). Por isso, comparações só são válidas entre execuções feitas nas mesmas condições, e é o que fazemos na Entrega 3.

O custo dominante com perda continuava sendo o timeout fixo: no SR-16 com 10 %, cada uma das 220 perdas custava 500 ms de espera, para um RTT típico de ∼25–35 ms.

1.3 Entrega 3 — RTT adaptativo

A Entrega 3 ataca exatamente esse custo: em vez de esperar 500 ms, o cliente estima o RTT por EWMA e usa $RTT_{est} + 4 \times DevRTT$ como timeout, aplicado a GBN e SR (o SAW permanece com timeout fixo, como referência). As dificuldades foram: (i) o RTT medido de um segmento retransmitido é ambíguo, o que já havia sido antecipado na Entrega 1 e foi resolvido com o algoritmo de Karn; (ii) o timeout agora muda ao longo da execução, então o prazo de cada `REQ` precisa ser guardado no momento do envio (`timeout_at_send`), tanto para o temporizador individual do SR quanto para o temporizador único da base no GBN; e (iii) para saber se um timeout foi necessário ou desnecessário, passamos a estimar as retransmissões espúrias (retransmissões menos DATA descartados pela simulação). A evolução, em resumo:

Tabela 3: Evolução do projeto.

Entrega	Implementação	Limitação medida	O que motivou a seguinte
1	SAW, timeout fixo	1 segmento por RTT (∼105–115 kbps de teto); perdas custam 500 ms	Mais segmentos em trânsito
2	GBN e SR, perda simulada	Com perda, GBN reenvia a janela toda; SR ainda espera 500 ms por perda	Timeout proporcional ao RTT real
3	Timeout adaptativo (EWMA + Karn)	Seções 2 e 3	—

2 RTT adaptativo — implementação

2.1 Fórmulas e regras implementadas

O estimador está em `update_rtt_estimator` (`comum.py`) e roda quando um segmento é aceito, com R = `rtt_ms` = instante de chegada do `DATA` − instante do último `REQ` daquele segmento:

$$\begin{aligned} DevRTT &= (1-\beta)\, DevRTT + \beta\, |R - RTT_{est}| \\ RTT_{est} &= (1-\alpha)\, RTT_{est} + \alpha\, R \\ RTO &= RTT_{est} + 4\, DevRTT \end{aligned}$$

- **Inicialização** (primeira amostra válida, como no TCP): $RTT_{est} = R$ e $DevRTT = R/2$. Até essa primeira amostra o timeout é o valor de `--timeout` (500 ms).
- **Desvio:** calculado com o RTT_{est} anterior à atualização, como no enunciado.
- **Timeout:** limitado a [`--min-rto` = 1 ms, 60 s] apenas para evitar RTO zero ou espera ilimitada.
- **Karn:** só amostras de segmentos sem retransmissão atualizam o estimador (`retransmissions == 0`), pois, após um reenvio, o `DATA` recebido pode ser resposta ao `REQ` original e o RTT medido a partir do último envio ficaria artificialmente curto. Pela mesma razão, o RTT médio dos resumos também usa só amostras Karn-válidas.

- **RTO guardado por envio:** `send_req` registra `timeout_at_send[seq]`. No SR, o prazo de cada segmento é `last_send[seq] + timeout_at_send[seq]`; no GBN, o temporizador único da base usa o RTO guardado no envio da base e é reiniciado quando a base avança ou é retransmitida. O `timeout_ms` do CSV é, portanto, o RTO vigente no envio do último `REQ` do segmento.

O CSV por segmento registra `rtt_ms`, `rtt_estimated_ms`, `dev_rtt_ms`, `timeout_ms`, `timeout_mode`, `alpha` e `beta`. Uma limitação assumida: não implementamos o *backoff* exponencial do TCP (dobrar o RTO após cada timeout); com perdas independentes e RTT estável isso não foi necessário, mas seria o primeiro ajuste em um caminho com congestionamento real.

Um ponto específico deste ambiente: com janela 16 o RTT medido inclui o enfileiramento dos `REQ` em voo (na Entrega 2, o RTT médio subiu de ∼24 ms para ∼32–34 ms com $N = 16$). O estimador mede esse RTT efetivo, que é justamente o que importa para decidir quando um `DATA` está atrasado.

2.2 Escolha de α e β

Os valores do TCP ($\alpha = 0{,}125$ e $\beta = 0{,}25$) foram pensados para a Internet em geral. Para decidir o que funciona melhor neste ambiente (RTT ∼22–35 ms, mediana ∼26–30 ms e picos de até ∼470 ms observados na Entrega 1), fizemos uma varredura $\alpha \times \beta$ com SR-16 e o arquivo small, em três níveis de perda simulada, com repetições e médias por combinação. Para cada combinação foram comparados:

- **throughput médio** (kbps), que resume o custo total das esperas;
- **retransmissões espúrias** = retransmissões − DATA descartados pela simulação. No SR cada descarte exige exatamente 1 retransmissão, então o excedente é timeout que estourou com o DATA ainda a caminho; esta é a medida do risco de um timeout agressivo demais;
- **timeout médio** (ms), que mostra quão folgado ou apertado o RTO ficou.

O esperado era um compromisso: α e β maiores acompanham mais rápido as variações do RTT, mas deixam o RTO mais nervoso; β menor aperta o RTO e reduz a espera por perda, mas aumenta o risco de timeout espúrio quando o RTT tem picos.

Tabela 4: Varredura $\alpha \times \beta$: SR-16, arquivo small, média de 2 execuções por combinação. Vazão em kbps; espúrias = retransmissões − DATA descartados, com perda de 0 / 5 / 10 %.

α	β	**0 %**	**5 %**	**10 %**	**Média**	**Espúrias (0 / 5 / 10 %)**
0,0625	0,125	923,0	1 198,4	991,2	1 037,5	2 845 / 920 / 700
0,0625	0,25	1 248,0	1 159,0	955,6	1 120,9	1 707 / 779 / 756
0,0625	0,5	1 006,6	1 078,6	917,6	1 000,9	3 590 / 1 388 / 707
0,125	0,125	1 040,3	1 177,6	959,9	1 059,3	2 465 / 942 / 830
0,125 (TCP)	0,25 (TCP)	1 466,9	1 156,8	940,6	1 188,1	1 906 / 1 128 / 904
0,125	0,5	1 544,5	1 136,0	986,8	1 222,4	1 202 / 1 182 / 1 006
0,25	**0,125**	**1 564,6**	1 176,3	**1 050,4**	**1 263,8**	1 408 / 1 154 / 930
0,25	0,25	1 528,5	1 120,8	820,8	1 156,7	1 596 / 1 524 / 1 942
0,25	0,5	1 484,9	1 210,7	836,7	1 177,4	1 723 / 1 356 / 2 124
0,5	0,125	1 505,1	1 114,6	825,6	1 148,4	1 863 / 2 035 / 3 192
0,5	0,25	1 417,9	1 174,5	872,9	1 155,1	1 834 / 1 761 / 2 682
0,5	0,5	1 489,3	1 171,5	932,4	1 197,7	1 872 / 1 768 / 1 555

Escolha: $\alpha = 0{,}25$ e $\beta = 0{,}125$. Essa combinação teve a maior vazão média nas três perdas (1 263,8 kbps), 6 % acima da dos valores do TCP ($\alpha = 0{,}125$ e $\beta = 0{,}25$: 1 188,1 kbps), e foi a melhor com 0 % (1 564,6 kbps) e com 10 % de perda (1 050,4 kbps). Com 5 % as combinações ficam próximas (1 079 a 1 211 kbps), dentro do que duas repetições por ponto permitem distinguir; por isso a escolha se apoia no conjunto das três perdas, não em um ponto isolado.

Dois efeitos aparecem nos dados (Figura 1). Primeiro, α maior aproxima o RTO do RTT: com 10 % de perda, o timeout médio cai de 41–44,5 ms ($\alpha = 0{,}0625$) para 32–36,5 ms ($\alpha = 0{,}5$). Segundo, aproximar

demais o RTO tem preço: $\alpha = 0{,}5$ com 10 % de perda deu as piores vazões (826–932 kbps) e até 3 750 retransmissões (3 192 espúrias com $\beta = 0{,}125$). No outro extremo, $\alpha = 0{,}0625$ reage devagar e ficou abaixo de 1 250 kbps com 0 % de perda, contra 1 418–1 565 kbps para $\alpha \geq 0{,}25$, embora tenha poucas espúrias com 10 %.

Os valores do TCP ficaram a 6 % do melhor neste ambiente, então o ganho de ajustar α e β é pequeno perto do ganho do adaptativo sobre o fixo (Seção 3) e incerto com duas repetições. Todas as combinações tiveram muitas retransmissões espúrias, inclusive com 0 % de perda (1 202 a 3 590 retransmissões para 2 048 segmentos), tema da Seção 3.3.

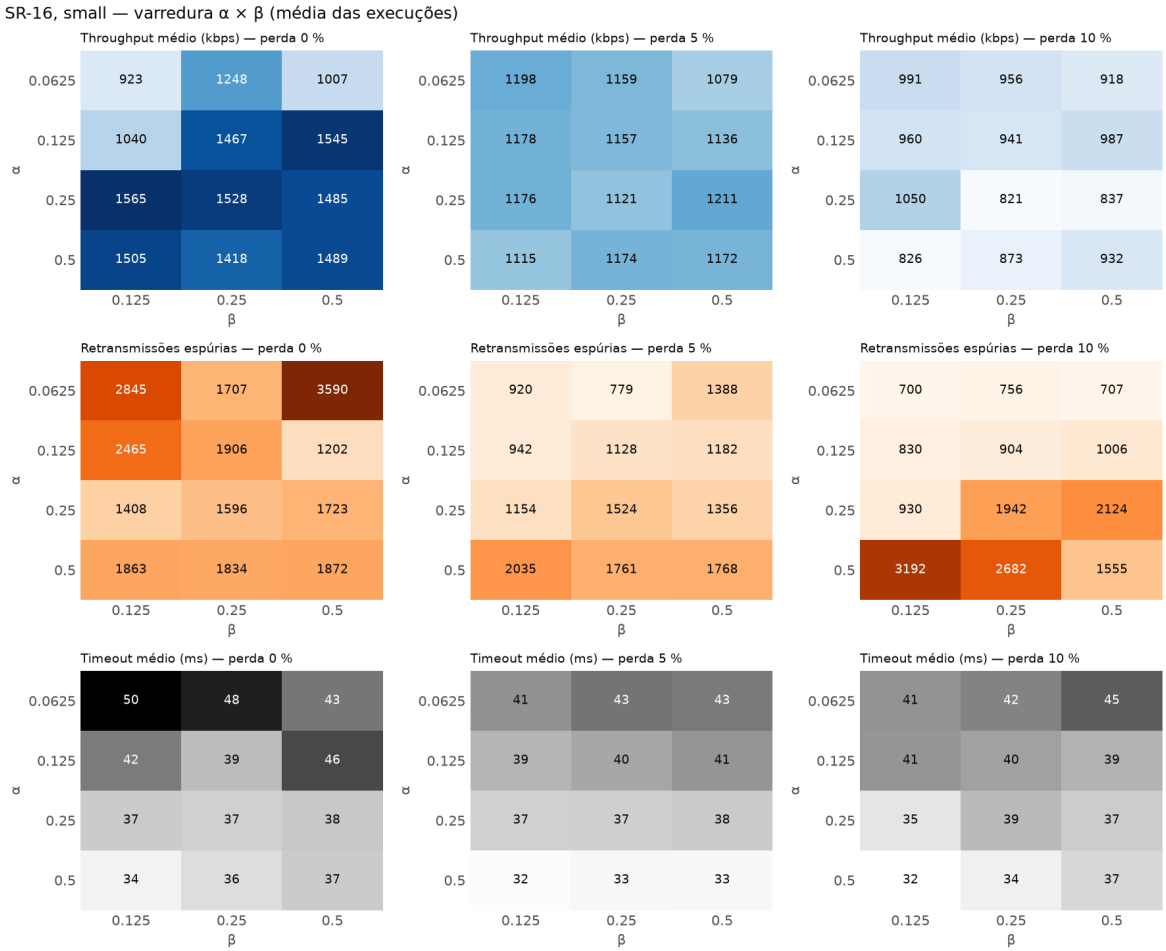


Figura 1: Mapa de calor da varredura $\alpha \times \beta$: throughput, retransmissões espúrias e timeout médio por perda simulada.

2.3 Convergência do estimador e evolução do timeout

As Figuras 2 e 3 mostram os dois gráficos pedidos, com $\alpha = 0{,}25$ e $\beta = 0{,}125$, para o SR-16 adaptativo com o arquivo large (20 480 segmentos, sem perda simulada; a mesma execução capturada no Wireshark, Seção 4). Em cada figura, o **painel de cima** é o RTT estimado × RTT medido por segmento e o **painel de baixo** é o timeout ao longo da transferência.

Convergência (Figura 3). Os 16 primeiros `REQ` saem com o timeout inicial de 500 ms. Quando chega o primeiro `DATA`, o estimador começa em $RTT_{est} = 24{,}0$ ms e $DevRTT = 12{,}0$ ms ($R/2$), e o timeout cai para 72 ms no segmento 16 e para ∼31 ms no segmento 40. A partir daí o $DevRTT$ continua diminuindo e o RTT_{est} acompanha os degraus do RTT medido (21–28 ms), que mudam quando o enfileiramento dos 16 `REQ` em voo muda. Com $\alpha = 0{,}25$ cada degrau é absorvido em poucas amostras.

Adaptação do timeout (Figura 2). Ao longo da transferência, o timeout segue o RTT estimado com folga de só 3–6 ms, porque o $DevRTT$ fica em torno de 1 ms (0,9 ms no fim). Quando o RTT

sobe num degrau, o timeout sobe junto; quando cai, o timeout cai. É o comportamento esperado do estimador, mas tem um efeito colateral visível nos dois painéis: as marcas pretas (10 176 segmentos com timeout) e os X laranja (RTT de segmentos retransmitidos, quase todos abaixo de 20 ms, medidos a partir do reenvio e ignorados pelo algoritmo de Karn). Cada pico de RTT acima da folga dispara timeouts em sequência, e como essas amostras longas são justamente as descartadas por Karn, o estimador não "aprende" com elas e o RTO continua apertado. As consequências estão na Seção 3.3, e a correção, na Seção 3.4.



Figura 2: SR-16 adaptativo, arquivo large, transferência inteira. Em cima: RTT estimado (EWMA) × RTT medido por segmento; em laranja, amostras de segmentos retransmitidos, ignoradas pelo estimador (Karn). Embaixo: timeout ao longo da transferência.

3 Comparação entre timeout fixo e adaptativo

3.1 Método

Comparamos o mesmo protocolo com timeout fixo (500 ms) e adaptativo (com os α e β escolhidos na Seção 2.2) sob perda simulada de 5 % e 10 %, além do caso sem perda como controle. Usamos GBN-16 e SR-16 (`rtt_adaptativo.py comparacao`), arquivo small, `--seed 42` e as mesmas condições de rede: as execuções fixo e adaptativo de cada cenário entraram na mesma rodada de experimentos, porque a rede varia de um dia para o outro (Seção 1.2). Os valores são médias de 2 repetições por cenário, todas com MD5 correto e `DONE ok=1`. Retransmissões espúrias são estimadas como retransmissões − DATA descartados pela simulação; no SR ela é quase exata com timeout fixo, mas subestima com o adaptativo (cópias duplicadas de reenvios espúrios também são descartadas pela simulação e entram nos descartes), e no GBN inclui os reenvios "de carona" do "volta N", então só é comparada entre fixo e adaptativo do mesmo protocolo.

Como referência, os valores com timeout fixo obtidos na Entrega 2 (uma execução, seed 42, outro dia) foram: SR-16 com 5 % → 260,7 kbps, 91 retransmissões, 32,2 s; SR-16 com 10 % → 153,7 kbps, 220 retransmissões, 54,6 s; GBN-16 com 5 % → 159,5 kbps, 1 512 retransmissões, 52,6 s; GBN-16 com 10 % → 70,3 kbps, 3 562 retransmissões, 119,3 s.

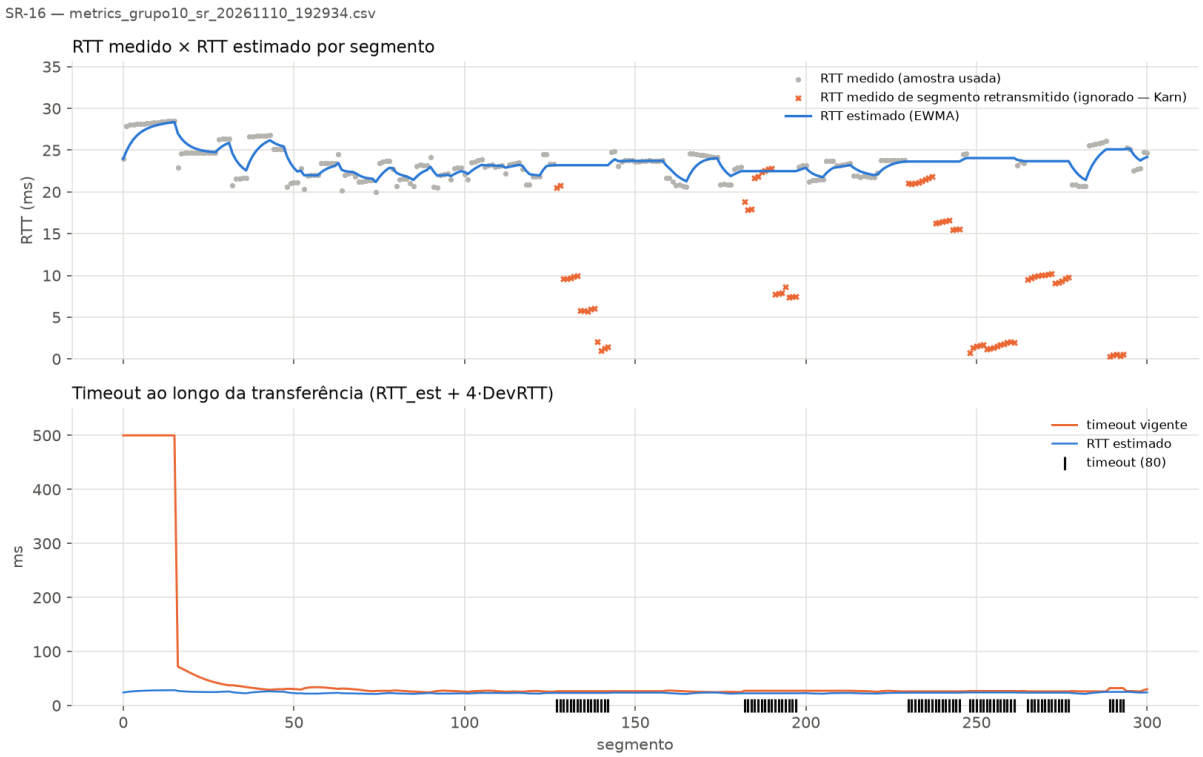


Figura 3: Zoom nos 300 primeiros segmentos. Em cima: convergência do RTT estimado a partir da primeira amostra. Embaixo: o timeout cai de 500 ms para ∼30 ms e passa a acompanhar as variações do RTT.

3.2 Resultados

Tabela 5: GBN-16 e SR-16, arquivo small, média de 2 execuções. Espúrias = retransmissões − DATA descartados pela simulação.

Protocolo	Perda	Timeout	Retrans-missões	Tempo (s)	Throughput (kbps)	Descartes simulados	Espúrias	Timeout médio (ms)
GBN-16	0 %	fixo	16	5,76	1 455,9	0	16	500
GBN-16	0 %	adaptativo	464	5,31	1 578,9	0	464	48,9
GBN-16	5 %	fixo	1 512	55,73	150,5	156	1 356	500
GBN-16	5 %	adaptativo	1 926	12,37	678,9	172	1 754	50,4
GBN-16	10 %	fixo	3 598	123,71	67,8	536	3 062	500
GBN-16	10 %	adaptativo	4 112	20,90	401,7	593	3 519	48,6
SR-16	0 %	fixo	10	6,10	1 411,8	0	10	500
SR-16	0 %	adaptativo	1 007	4,83	1 737,9	0	1 007	39,3
SR-16	5 %	fixo	105	33,77	248,5	91	14	500
SR-16	5 %	adaptativo	1 466	6,92	1 221,0	150	1 316	35,6
SR-16	10 %	fixo	242	56,34	148,9	220	22	500
SR-16	10 %	adaptativo	1 343	8,46	991,2	326	1 017	37,9

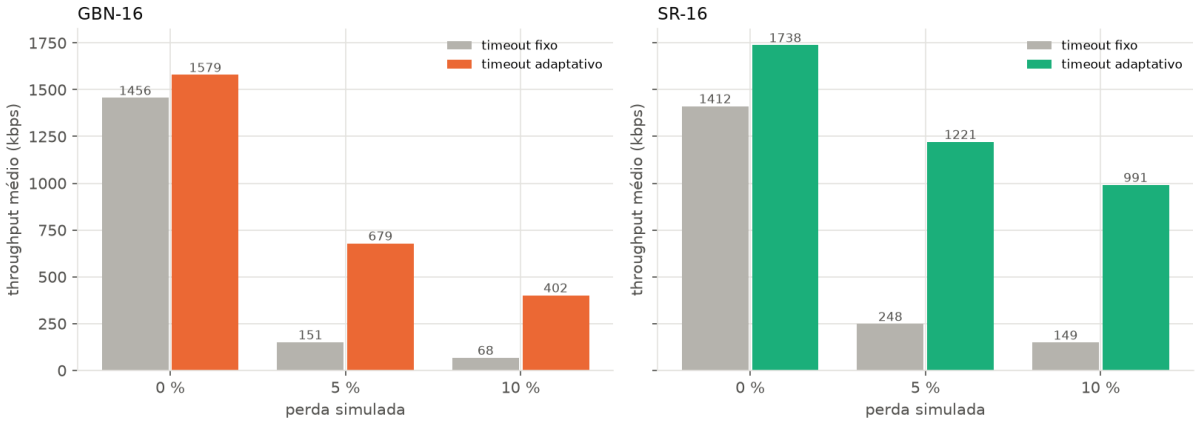


Figura 4: Throughput médio com timeout fixo e adaptativo, por perda simulada.

Com perda, o adaptativo cortou o tempo de transferência em 78 % a 85 %. No SR-16, o tempo caiu de 33,8 s para 6,9 s com 5 % de perda (−80 %) e de 56,3 s para 8,5 s com 10 % (−85 %), o que multiplica a vazão por 4,9 e por 6,7. No GBN-16, a queda foi de 55,7 s para 12,4 s (−78 %) e de 123,7 s para 20,9 s (−83 %), com vazão ×4,5 e ×5,9. Os resultados com timeout fixo repetem os da Entrega 2 (por exemplo, GBN-16 com 5 %: 1 512 retransmissões nas duas). Mesmo adaptativo, o GBN-16 com 10 % (401,7 kbps) ficou abaixo do SR-16 (991,2 kbps).

Sem perda simulada, a diferença de tempo é pequena (−21 % no SR e −8 % no GBN), mas o número de retransmissões sobe: o adaptativo fez 1 007 retransmissões no SR e 464 no GBN, contra 10 e 16 com o fixo. Com perda elas também aumentaram: no SR, de 105 para 1 466 (5 %) e de 242 para 1 343 (10 %); no GBN, 27 % e 14 % a mais. O adaptativo ganhou em tempo e perdeu em retransmissões, compromisso discutido na Seção 3.3.

3.3 Quando o adaptativo vence e quando o fixo basta

Com perda, o adaptativo compensa muito; sem perda, o fixo basta, e o preço do adaptativo são as retransmissões espúrias. Os pontos abaixo usam os números da Tabela 5.

Onde o adaptativo tem mais vantagem: quando há perda. Cada perda custa 500 ms ao timeout fixo, enquanto o timeout médio do adaptativo ficou em 36–50 ms, de 10 a 14 vezes menor. A economia cresce com a perda: 26,9 s (SR) e 43,4 s (GBN) com 5 %, e 47,9 s (SR) e 102,8 s (GBN) com 10 %. O efeito é maior no GBN, onde os timeouts se somam (Seção 1.2): com 10 % de perda, a vazão foi de 67,8 para 401,7 kbps.

Quando o fixo é suficiente: sem perda. Sem perda simulada, fixo e adaptativo diferem só pelas poucas perdas naturais da rede (10 retransmissões no SR e 16 no GBN com o fixo, cada uma custando até 500 ms), o que explica a diferença de 21 % e 8 % no tempo. Se a rede não perdesse pacotes, ou se o tempo de recuperação não importasse, o fixo de 500 ms seria suficiente, mais simples e sem retransmissões desnecessárias.

O preço: retransmissões espúrias. Com perda zero, o adaptativo fez 1 007 retransmissões no SR (805 segmentos com timeout) e 464 no GBN, contra 10 e 16 com o fixo. O timeout médio (39 ms no SR) ficou muito perto do RTT médio (28 ms), e os picos de RTT (de até ∼470 ms na Entrega 1, além do enfileiramento com janela 16) o ultrapassam. O mesmo aparece em todas as combinações da varredura. No SR um espúrio custa um `REQ` e um `DATA` duplicado, que o cliente descarta; no GBN custa até N retransmissões de uma vez. Aqui isso não impediu o ganho de tempo, mas em uma rede com banda disputada esses reenvios ocupariam o caminho e o servidor.

Uma explicação provável é o algoritmo de Karn sem backoff: as amostras de RTT longo são justamente as que sofrem timeout espúrio e retransmissão, então são descartadas pelo estimador, e o RTO tende a ficar baixo demais (Figura 2). Os testes naturais seriam dobrar o RTO após cada timeout, como o

TCP, ou subir o piso `--min-rto`, que já existe no cliente; testamos o piso na Seção 3.4.

Em resumo: neste ambiente (RTT de ∼28 ms e perda alta), o adaptativo foi 4,5 a 6,7 vezes mais rápido que o fixo e vale a pena quando há perda, preferencialmente no SR, que tolera melhor um RTO agressivo (991,2 kbps contra 401,7 kbps do GBN com 10 %); sem perda, o fixo é suficiente.

3.4 Teste do RTO mínimo

No arquivo large e sem perda, o efeito das espúrias passa a dominar: o SR-16 adaptativo fez 16 749 retransmissões e ficou *abaixo* do fixo (1 690 contra 2 067 kbps, Tabela 8). O TCP real evita isso com um RTO mínimo (200 ms a 1 s). Repetimos o SR-16 adaptativo com pisos de 50 e 100 ms (`--min-rto`; uma execução por cenário, 20:13–20:15 de 10/11):

Tabela 6: SR-16 adaptativo com RTO mínimo: throughput (kbps) / retransmissões.

Cenário	Sem piso (1 ms)	Piso 50 ms	Piso 100 ms	Fixo 500 ms
large, sem perda	1 690 / 16 749	2 120 / 2 183	**2 237 / 794**	2 067 / 13
small, sem perda	1 738 / 1 007	**2 190 / 162**	2 144 / 81	1 412 / 10
small, 5 %	**1 221 / 1 466**	1 201 / 317	824 / 175	249 / 105
small, 10 %	**991 / 1 343**	945 / 453	593 / 300	149 / 242

O piso resolve o conflito entre os dois casos: sem perda, ele elimina a maior parte das espúrias (16 749 → 794 no large) e o adaptativo passa a superar o fixo; com perda, um piso alto atrasa a recuperação das perdas reais (100 ms: 593 contra 991 kbps com 10 %). O piso de 50 ms ficou perto do melhor em todos os cenários (3 % acima do fixo no large e 55 % no small sem perda; 4,8× acima com 5 %).

4 Análise Wireshark

Capturamos a transferência do SR-16 adaptativo com o arquivo large (10/11/2026, 19:28:45–19:29:34) com o filtro `udp and host 137.131.178.229`: 94 948 pacotes, dos quais 37 229 REQs (= 20 480 segmentos + 16 749 retransmissões, exatamente o total do CSV).
Arquivo da captura: `captura_sr16_adaptativo_large.pcapng`.

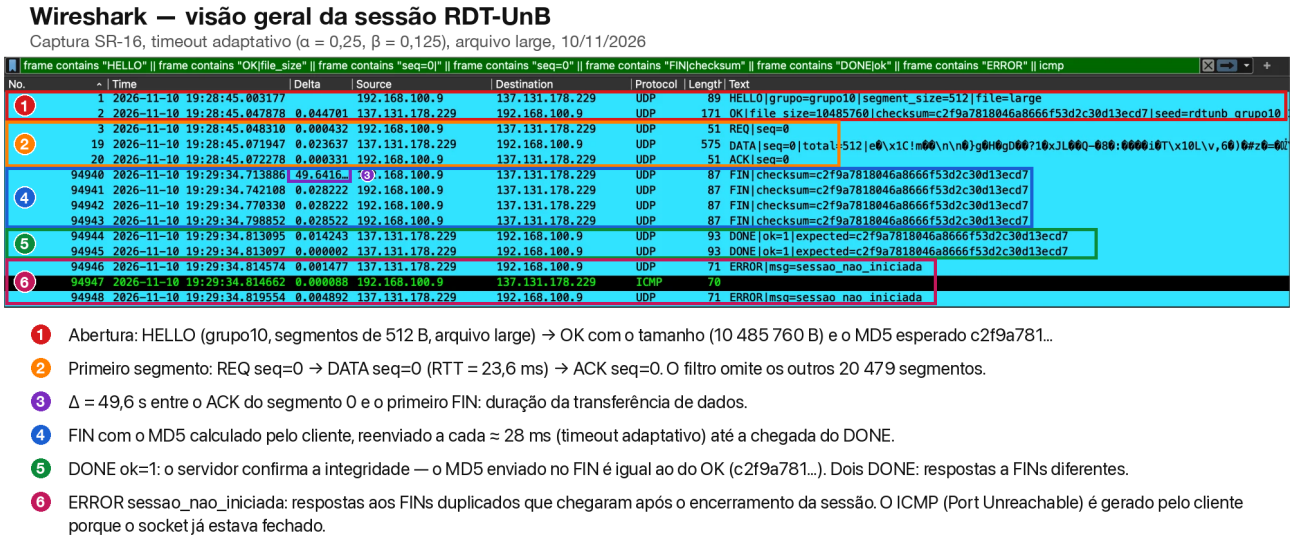
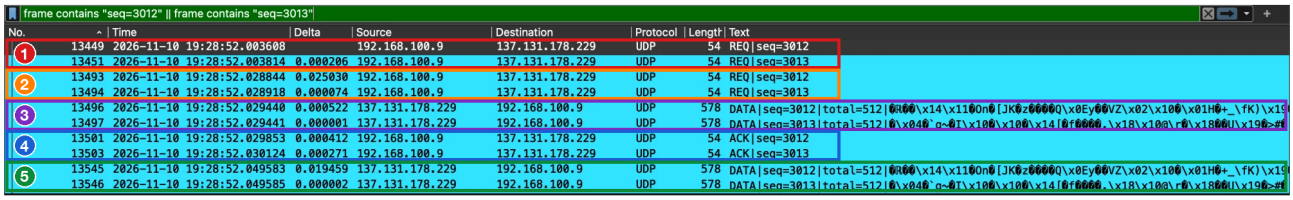


Figura 5: Screenshot 1 — visão geral da sessão: HELLO → OK → REQ/DATA/ACK → FIN → DONE.

Wireshark — retransmissões espúrias: dois eventos independentes

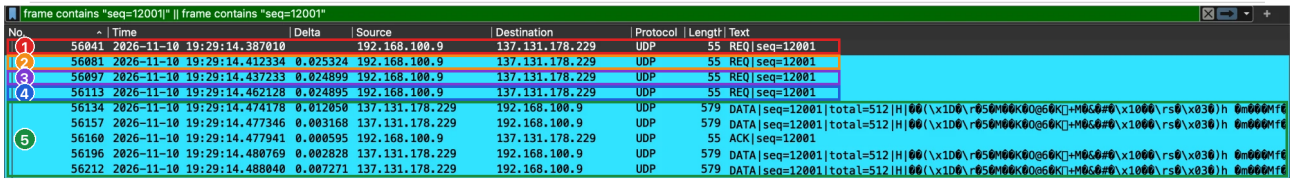
Captura SR-16, timeout adaptativo (α = 0,25, β = 0,125), arquivo large, 10/11/2026

Evento 1 — segmentos 3012 e 3013, t ≈ 7,0 s (recv_time_ms = 6 981,8)



1. REQ originais dos segmentos 3012 e 3013 (t = 0, frame 13449).
2. O timeout adaptativo do momento (24,1 ms, no CSV) expira e os REQ são reenviados em t = 25,2 ms.
3. Os DATA chegam em t = 25,8 ms, 0,6 ms depois do reenvio: são as respostas aos REQ originais. O RTT real (≈ 25,8 ms) ficou pouco acima do timeout — nenhum pacote foi perdido.
4. ACKs. O CSV registra RTT de 1,0 ms (medido a partir do reenvio): é a ambiguidade que o algoritmo de Karn evita, descartando a amostra de segmentos retransmitidos no estimador.
5. DATA duplicados, em resposta aos REQ reenviados: retransmissão espúria, 2 × 512 B desperdiçados. Repetido ao longo da transferência, isso explica as 16 749 retransmissões do SR-16 adaptativo.

Evento 2 — segmento 12001, t ≈ 29,4 s (recv_time_ms = 29 430,5)



1. REQ original do segmento 12001 (frame 56041, t = 0 → 19:29:14,387010).
2. Nenhum DATA chega dentro do timeout adaptativo do momento (timeout_ms = 24,26 no CSV): 1° reenvio em t = 25,3 ms.
3. 2° reenvio em t = 50,2 ms (Δ = 24,9 ms).
4. 3° reenvio em t = 75,1 ms (Δ = 24,9 ms). O intervalo entre os REQ é o timeout vigente (24,26 ms + latência de escalonamento).
5. Chegam 4 DATA, um para cada REQ, sem nenhuma perda: o primeiro em t = 87,2 ms, ou seja, o RTT do REQ original subiu de ~22 ms para ~87 ms (pico de atraso). O cliente aceita um DATA e envia o ACK (frame 56160); os outros 3 são duplicatas descartadas. No CSV: timeout_ms = 24,26 e retransmissions = 3. O rtt_ms = 15,84 ms foi medido a partir do último reenvio: amostra ambígua, ignorada pelo estimador (algoritmo de Karn).

Figura 6: Screenshot 2 — dois eventos de retransmissão independentes, em $t \approx 7{,}0\,\text{s}$ (segmentos 3012 e 3013) e $t \approx 29{,}4\,\text{s}$ (segmento 12001), com os tempos da coluna `recv_time_ms` do CSV.

Nos dois eventos o REQ é reenviado depois de um intervalo próximo do timeout adaptativo vigente (24,1 ms e 24,26 ms no CSV) e todos os DATA acabam chegando: são retransmissões **espúrias**, causadas por picos de atraso e não por perda. No segmento 12001 o RTT do REQ original foi de 87,2 ms, o que gerou três reenvios e três DATA duplicados.

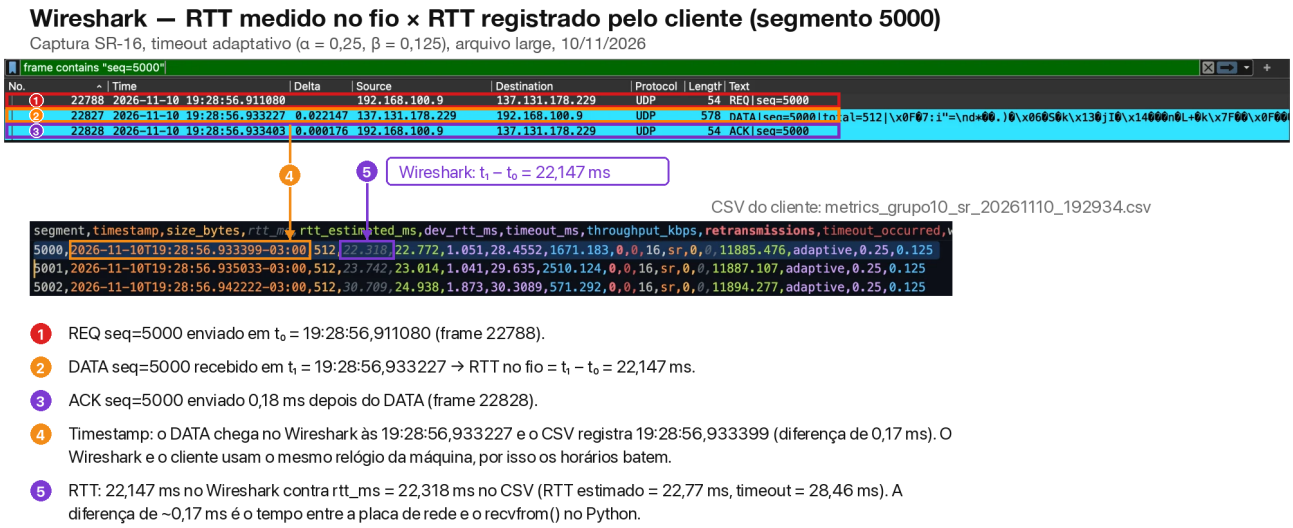


Figura 7: Screenshot 3 — correlação entre o Wireshark e a linha do segmento 5000 no CSV.

RTT no Wireshark × RTT no CSV. No segmento 5000, o RTT no fio foi 22,147 ms e o do CSV 22,318 ms (diferença de 0,17 ms); o timestamp do DATA é 19:28:56,933227 no Wireshark e 19:28:56,933399 no CSV. Os horários batem porque o Wireshark e o cliente usam o mesmo relógio da máquina. Para não depender de um único segmento, extraímos do `.pcapng` (com `tshark`) o tempo entre o REQ e o DATA de todos os segmentos com um único REQ e comparamos com o `rtt_ms` do CSV:

Tabela 7: RTT no fio (Wireshark) × RTT registrado pelo cliente, 10 304 segmentos sem retransmissão.

	Média (ms)	Mediana (ms)
Wireshark (DATA − REQ)	22,51	22,39
CSV (`rtt_ms`)	23,49	23,52
Diferença CSV − Wireshark	0,98	0,77

Os valores são próximos: a diferença fica entre 0,19 e 2,43 ms em 90 % dos segmentos e é *sempre positiva* (mínimo 0,08 ms), como esperado. O Wireshark marca o instante em que o pacote passa pela placa de rede; o cliente marca depois do `recvfrom()` no Python. Com 16 segmentos em voo, os DATA chegam em rajadas e o cliente processa um de cada vez (ACK, linha do CSV, novo REQ), então os últimos de cada rajada esperam alguns ms no buffer do socket. O segmento 5000 caiu num momento sem fila (0,17 ms).

5 Visão integrada — comparação final de todos os protocolos

Para comparar os protocolos nas mesmas condições de rede, rodamos todos com o arquivo large na mesma janela de tempo (10/11/2026, 19:23–19:29), sem perda simulada. Todos terminaram com MD5 `c2f9a7818046a8666f53d2c30d13ecd7` e `DONE ok=1`.

Tabela 8: Visão integrada: melhor cenário de cada protocolo, arquivo large (10 MB), sem perda simulada.

Protocolo	Janela	Timeout	Throughput (kbps)	Retransmissões	Tempo total (s)
SAW	1	fixo	150,55	3	557,18
GBN	16	fixo	2 014,30	96	41,65
GBN	16	adaptativo	2 092,78	4 032	40,08
SR	16	fixo	2 067,29	13	40,58
SR	16	adaptativo	1 690,49	16 749	49,62
SR	16	adapt. + piso 100 ms*	**2 237,45**	794	**37,49**

* Execução extra (20:15), Seção 3.1.

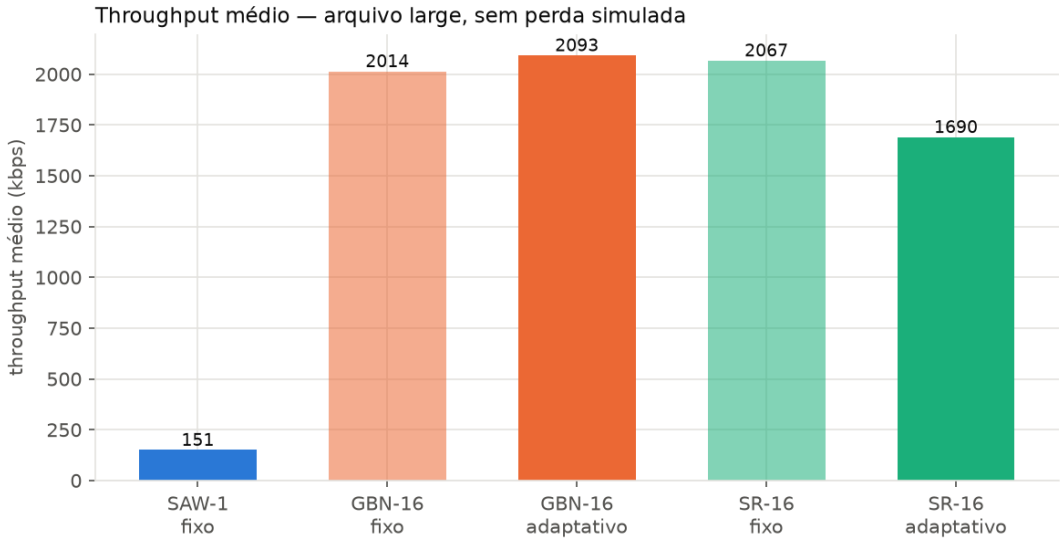


Figura 8: Throughput médio de todos os protocolos, arquivo large, sem perda simulada.

A tabela conta a história do projeto: a janela multiplicou a vazão por ∼14 (SAW 151 → GBN/SR ∼2 000–2 100 kbps, tempo de 9 min para 40 s). O SAW da Entrega 1 com o mesmo arquivo levou 965,9 s (86,8 kbps, 465 retransmissões naturais, 13/10/2026); a diferença para os 557 s daqui é só a rede, mais estável no dia 10/11 (RTT médio de 35,6 contra 26,9 ms e 3 retransmissões). Sem perda, GBN e SR ficam dentro de 4 % um do outro e o timeout adaptativo sem piso não ajuda; os ganhos do SR e do timeout adaptativo aparecem com perda (Tabela 9).

5.1 Todas as implementações com perda

A Tabela 9 junta as três entregas nos cenários com perda simulada (arquivo small, `--seed 42`). As linhas de timeout fixo são da Entrega 2 (26/10/2026); as de timeout adaptativo, da Entrega 3 (10/11/2026, média de 2 execuções).

Tabela 9: Todas as implementações, arquivo small: throughput (kbps) / retransmissões / tempo total (s).

Protocolo	Timeout	Sem perda	5 % de perda	10 % de perda
SAW-1	fixo (E2)	167 / 0 / 50,2	88 / 91 / 95,0	52 / 220 / 161,3
GBN-16	fixo (E2)	2 026 / 0 / 4,1	159 / 1 512 / 52,6	70 / 3 562 / 119,3
GBN-16	adaptativo (E3)	1 579 / 464 / 5,3	679 / 1 926 / 12,4	402 / 4 112 / 20,9
SR-16	fixo (E2)	1 901 / 0 / 4,4	261 / 91 / 32,2	154 / 220 / 54,6
SR-16	adaptativo (E3)	1 738 / 1 007 / 4,8	**1 221 / 1 466 / 6,9**	**991 / 1 343 / 8,5**

- **Sem perda, a janela é o que importa:** GBN-16 e SR-16 têm 11–12× o throughput do SAW, e o timeout quase não interfere. Na Entrega 2, a vazão cresceu com a janela (SAW 167 → ∼650 com $N = 4$ → ∼1 200 com $N = 8$ → ∼2 000 kbps com $N = 16$).
- **Com perda e timeout fixo, o GBN quase não ganha do SAW** (10 %: 70 contra 52 kbps): cada timeout reenvia a janela inteira (3 562 retransmissões para as mesmas 220 perdas que o SR retransmite uma a uma) e trava a transferência por 500 ms. O SR continua andando enquanto espera, e com $N = 4/8/16$ foi de 84 para 110 e 154 kbps; o GBN ficou em 64–70.
- **O timeout adaptativo multiplica o ganho do SR:** com 10 % de perda, o SR-16 adaptativo (991 kbps) mantém 57 % do próprio desempenho sem perda e 19× acima do SAW. O GBN também melhora (402 kbps), mas continua limitado pelos reenvios em bloco.
- **Reprodutibilidade:** rodando de novo o timeout fixo em 10/11, com a mesma seed, os resultados com perda ficaram a menos de 6 % dos da Entrega 2 (GBN-16 10 %: 67,8 contra 70,3 kbps; SR-16 5 %: 248,5 contra 260,7 kbps). Sem perda, a diferença é maior (SR-16: 1 412 contra 1 901 kbps), porque aí o resultado depende só da rede no dia.

6 Conclusão

Qual protocolo teve melhor desempenho? O **SR-16**. Sem perda, GBN-16 e SR-16 empatam (∼2 000–2 100 kbps, 14× o SAW). Com perda, o SR é claramente melhor: retransmite só o segmento perdido, enquanto o GBN reenvia a janela inteira (com 10 % de perda e timeout adaptativo, SR-16 991 kbps contra GBN-16 402 kbps; 1 343 contra 4 112 retransmissões). O melhor resultado absoluto no arquivo large foi o SR-16 adaptativo com piso de 100 ms (2 237 kbps, 37,5 s).

O RTT adaptativo valeu a complexidade? Sim, mas com uma condição. Com perda, foi a mudança de maior impacto do projeto: 4,5–6,7× mais throughput que o fixo (Tabela 5), porque cada perda passou a custar ∼1–2 RTTs em vez de 500 ms. Sem perda e com RTT estável, a fórmula pura do livro deixou o timeout colado no RTT e piorou o SR no arquivo large (1 690 contra 2 067 kbps). A complexidade só compensa por completo junto com um RTO mínimo: com piso de 50 ms o adaptativo ficou perto do melhor em todos os cenários testados (Tabela 6).

O que faríamos diferente.

- Registrar no CSV o RTT medido a partir do *primeiro* REQ, além do último, para não depender só de Karn para tratar a ambiguidade.
- Rodar os experimentos com antecedência, como pegamos os dados somente do último dia, pode ser que tenhamos perdido uma rede mais estável. Na primeira rodada do arquivo large (10/11, 18:32–18:47) pegamos a rede degradada (o SAW levou 1 114 s, com 772 retransmissões naturais, contra 557 s e 3 retransmissões na rodada das 19:23) e precisou ser refeita.

Anexo A — CSVs e demais arquivos (links)

Todos os arquivos estão no repositório público `github.com/annalujansen/tr2-rdt-unb` (tag `entrega3`). Os CSVs por segmento seguem o padrão `metrics_grupo10_<protocolo>_<AAAAMMDD_HHMMSS>.csv`, um por execução, numa pasta por cenário; cada conjunto tem um `resumo*.csv` com uma linha por execução (tempo, RTT, throughput, retransmissões, MD5).

- **Entrega 1** — `entrega1/dados/`: SAW com small, medium e large (13/10/2026).
- **Entrega 2** — `entrega2/dados/`: 21 cenários (SAW, GBN e SR; janelas 4, 8 e 16; perdas 0, 5 e 10 %) e `tabela_comparativa.csv`.
- **Entrega 3** — `entrega3/dados/`:
  - `varredura/` — varredura $\alpha \times \beta$ (Seção 2.2);
  - `comparacao/` — timeout fixo × adaptativo (Seção 3);
  - `rto_minimo/` — RTO mínimo (Seção 3.1);
  - `large/` — arquivo large, tabela integrada e captura (Seções 4 e 5);

– `large_rodada1/` — primeira rodada do large, com a rede degradada.

CSV das Figuras 2–7: `metrics_grupo10_sr_20261110_192934.csv`.

Anexo B — Checksum MD5 dos arquivos recebidos

O cliente calcula o MD5 do arquivo remontado e o envia no `FIN`; o servidor compara com o MD5 informado no `OK` e responde `DONE|ok=1`. Em todas as execuções, `md5_ok = 1` e `done_ok = 1`. O conteúdo gerado pelo servidor mudou entre as entregas (por isso o MD5 do mesmo arquivo difere), mas em cada sessão o MD5 recebido bateu com o informado no `OK` (Figura 5).

Entrega	Protocolos	Timeout	Arquivo	MD5 recebido (= esperado)
1	SAW	fixo	small	`f989450f6ca695013ceb6bd3a5f6417c`
1	SAW	fixo	medium	`2593bb16a3f52e31a7b8773450e8da8b`
1	SAW	fixo	large	`5623bc385a7b296e6c57d99290c03c0f`
2	SAW, GBN-4/8/16, SR-4/8/16	fixo	small	`f989450f6ca695013ceb6bd3a5f6417c`
3	GBN-16, SR-16	fixo, adapt.	small	`669af092d18cc3f9a060e9d1fb630d64`
3	SAW, GBN-16, SR-16	fixo e/ou adapt.	large	`c2f9a7818046a8666f53d2c30d13ecd7`